



Data Driven Education

Discussion Forum Subteam

Presentation 2 - 03/12/2025

Meet the Team



Bo Zimpel
Education Researcher



Zilu Zhu
Project Manager/Backend Developer



Diya Jain
Machine Learning Engineer



Saloni Agshiker
Data Scientist



Ethan Lu
Data Engineer

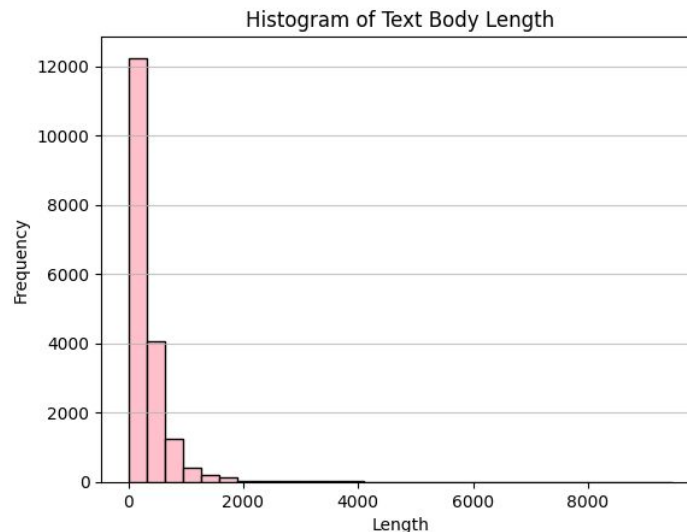
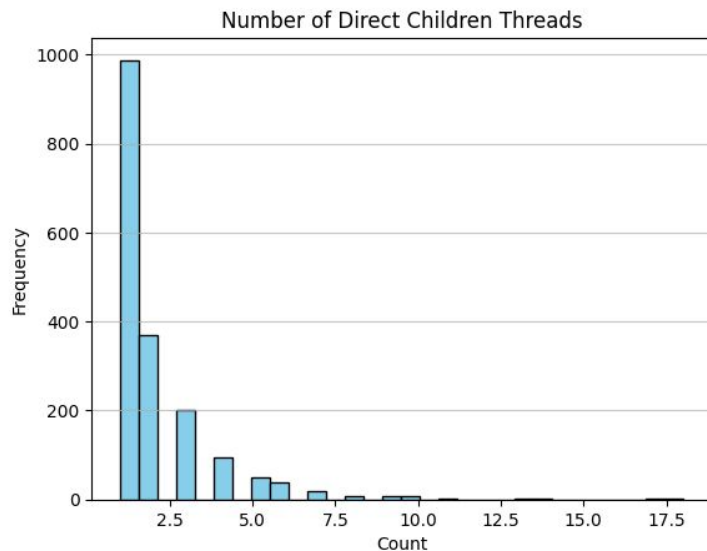
Introduction

- Online Discussion platform is crucial in modern education.
- **Ed Discussion:** a widely used online discussion forum
 - enables students and instructors to interact
 - discuss course materials asynchronously
- Questions proposed:
 - How is Online Discussion Forums different?
 - What to do to better utilize Online Discussion platforms?
 - Improve the whole education experience
- Progress:
 - Topic Modeling
 - RAG and LLMs



Dataset Analysis Progress

- 3 Dataset
 - Number of entries: 9188 + 9284 + 2197
- 13 Courses:
 - MGT1000; CS6035; CS6400; MGT6754; CS7641; ISYE6420; ISYE6402; ISYE6669; PUBP6501; CSE6250; CS1301
- Terms span from 2016 to 2020



Literature Review

Literature Review:

Goals:

- Find replicable NLP models for sentiment analysis and topic modeling
- Comparing traditional NLP (TF-IDF, LDA) for sentiment analysis and topic modeling vs deep learning (BERT)

Key NLP models

Model	Use Case	Pros	Cons
BERT	Sentiment classification	High accuracy, contextual understanding	Computationally expensive
LDA	Topic Modeling	Finds discussion themes	Topics may be vague
TF-IDF+ Logistic Regression	Baseline sentiment classifier	Simple, interpretable	Doesnt capture deep meaning
LSTM	Sentiment analysis	Captures sequence relationships	Needs large training data

Summary of Body of Literature

Liu et al. (2022) – Fine-tuned BERT-CNN for sentiment classification & engagement tracking in MOOC discussions.

- “BERT outperforms traditional ML models in student sentiment classification”

Jelodar et al. (2020)- Used LDA+ LSTM to analyze discussion trends in online learning

- “LDA is effective for topic modeling but struggles with contextual understanding which BERT addresses”

Farshad et al. (2024)- applied Gradient Boosting Models (GBM) to predict student engagement in project-based learning.

- We can compare GBM to deep learning approaches like BERT for engagement prediction

Zou et al. (2021) - Used social network analysis + BERT to evaluate student interactions in MOOC discussion

- “Social presence correlates with learner engagement, BERT enhances predictive accuracy,”
we can integrate BERT-based analysis into our dashboard to monitor discussion engagement

Data Science

Goals & Timeline for the Semester

- Explore **text-analysis models** for topic modeling & sentiment analysis
 - **LDA** - Latent Dirichlet Allocation - probabilistic model that identifies common topics in docs
 - **BERT** - Google's NLP model that can learn a wide breadth of tasks relating to language
- Learn **data cleaning & processing** techniques
- Revamp the **dashboard** from last semesters - make it interactive and informative for instructors

Now!



Feb 3-17

Feb 17 - March 3

March 3-24

March 24 - April 7

April 7-19

- Research text-analysis models
- Clean & pre-process data

- Create TF-IDF & LDA models
- Process & analyze results

- Fine-tune BERT
- Clean & pre-process dataset

- Analyze & compare results from both models

- Design & create dashboard using React

Current Progress: Data Cleaning, Structure & EDA

3-Step Data Wrangling Process:

- Used Pandas, NLTK, Gensim
- 1) Initial Merge + Clean
 - a) Combine comment and thread .csv files
 - b) Create functions to remove punctuation, common stopwords, and proper names
- 2) Thorough Scrub
 - a) Lemmatize data for improved text analysis
 - b) Remove “short-stumped” data points
- 3) Filter for Different Use-Cases
 - a) Create individual .csv files for each course
 - b) Merged all data points into one singular data

```
def remove_punctuation(text):  
    for char in text:  
        if char in string.punctuation:  
            text = text.replace(char, '')  
    return text
```

```
combined_df['body'] = combined_df['body'].apply(lambda x:remove_punctuation(x))
```

```
def stopwords_clean(text):  
    word_list=[]  
    for word in gensim.utils.simple_preprocess(text):  
        if word not in stopwords_list:  
            word_list.append(word)  
    final_list = ' '.join(word_list)  
    return final_list
```

```
combined_df['stopword_clean'] = combined_df['body'].apply(lambda x:stopwords_clean(x))  
print(combined_df['stopword_clean'].head(100))  
combined_df.to_csv('combined_forum_cleaned.csv', index=False)
```

Taking a
Closer Look at
Our Data...

42.7%

of the ~15k entries
are from CS1301

102

Different Courses
Polled From

EDA #1:

- Combine datasets
- Remove punctuation, stopwords & names

EDA #2:

- Lemmatize data
- Create new .csv file

EDA #3:

- Filter by course name
- Create individual files

Current Progress: Data Cleaning, Structure & EDA cont.

```
"Question about the syllabus due dates. It seems like week 1 should  
end Feb 19 (week 1: Feb 12 - Feb 19), however the due date for quiz 1  
shows Feb 21. I just want to confirm the due date. Thanks, PERSON"
```

remove punct.

all lowercase

remove stopwords

remove names

remove #'s

```
question syllabus due dates seems week end feb week  
feb feb however due date quiz shows feb confirm due date
```

Current Progress: Data Cleaning, Structure & EDA cont.

Encountered Challenges

- Using spaCy vs. NLTK vs. Presidio Analyzer for **NLP text processing**
- Merging same-name, diff.-year courses
- Fine-tuning dataset to be compatible with **various models** (i.e. LDA, TF-IDF, BERT)

Optimal Solutions

- Imported each library, implemented similar functions & compared space-time complexity → **NLTK** is the winner
- Created a function that takes in a general course name arg. as a **filter**
- **Communicated** clearly with DS subteam on logistics, priorities & strategies

```
[ 'course-v1:GTx+CS1301xII+1T2018' 'course-v1:GTx+ISYE6501x+1T2018'
  'course-v1:GTx+CSE6040x+2T2019' 'course-v1:GTx+ISYE6501x+3T2018'
  'course-v1:GTx+CS1301xII+3T2018' 'course-v1:GTx+ISYE6501x+3T2017'
  'course-v1:GTx+ISYE6501x+2T2019b' 'course-v1:GTx+ICT100x+3T2017'
```

```
def filter_by_course(course_name):

    # Filter rows where course_id contains any of the keywords
    filtered_df = df[df["course_id"].astype(str).str.contains(course_name, na=False)]

    # Save to a new CSV file
    filtered_df.to_csv(f'{course_name}_filtered.csv', index=False)
```

Current Progress: TF-IDF & LDA Models

```
data = pd.read_csv('sample_data/abcnews-date-text.csv');
data_text = data[['headline_text']]
data_text['index'] = data_text.index
documents = data_text
```

```
print("Number of docs: ", len(documents))
print(documents[:5])
```

Pandas for data manipulation

Show hidden output

```
processed_docs = documents['headline_text'].str.split()
dictionary = gensim.corpora.Dictionary(processed_docs) #tokenizes each headline (makes each headline into an array with each word as a separate element)
count = 0
for k, v in dictionary.iteritems(): #prints first 10 entries in the dictionary w/ their unique ID
    print(k, v)
    count += 1
    if count > 10:
        break
```

Maps each comment to unique ID in dictionary

Show hidden output

```
[ ] dictionary.filter_extremes(no_below=15, no_above=0.5, keep_n=100000) #filter tokens that appear in less than 15 docs or more than 50% of the docs
```

Filters comments below/above certain frequency

```
[ ] #creates a bag of words (BoW) representation for each doc in processed_docs using the dictionary
# BoW format transforms a document (list of words) into a list of (word_id, frequency) tuples, where frequency is the # of times the word appears in the doc
bow_corpus = [dictionary.doc2bow(doc) for doc in processed_docs]
bow_corpus[4310] #prints the BoW representation for document 4310
```

Gensim for NLP & topic modeling, calculates frequency of each word in comment

Show hidden output

```
[ ] bow_doc_4310 = bow_corpus[4310]
for i in range(len(bow_doc_4310)):
    print("Word {} (\"{}\") appears {} time.".format(
        bow_doc_4310[i][0],
        dictionary[bow_doc_4310[i][0]],
        bow_doc_4310[i][1]))
    # unique ID of word
    # actual word from dictionary
    # frequency of word in doc
```

Word 218 ("govt") appears 1 time.
Word 319 ("group") appears 1 time.
Word 801 ("local") appears 1 time.
Word 1922 ("wants") appears 1 time.
Word 5732 ("compulsory") appears 1 time.
Word 5733 ("ratepayers") appears 1 time.
Word 5734 ("voting") appears 1 time.

Show hidden output

```
[ ] tfidf = models.TfidfModel(bow_corpus) # creates a TF-IDF model from the bow_corpus
corpus_tfidf = tfidf[bow_corpus] # transforms the entire BoW corpus into TF-IDF weighted form
for doc in corpus_tfidf:
    word_tfidf = [(dictionary[id], score) for id, score in doc]
    pprint(word_tfidf)
    #pprint(doc) # pretty-prints the first transformed doc, where each tuple consists of (word ID, TF-IDF score)
    break
```

Creates TF-IDF model to measure importance of word

Show hidden output

```
[ ] lda_model = gensim.models.LdaMulticore(bow_corpus, num_topics=10, id2word=dictionary, passes=2, workers=2) # training LDA with BoW corpus
# in this case, common words (eg: "the", "is") may dominate => problematic
```

Trains LDA model

Current Progress: Results

TF-IDF Model - Prints each word in a comment, along with its importance score.

```
[('answer', 0.2776368554739517),  
 ('corrected', 0.34482969409266945),  
 ('didnt', 0.22214677208071149),  
 ('enter', 0.29263638009262855),  
 ('first', 0.16575642832827267),  
 ('instruction', 0.23265590071799852),  
 ('kind', 0.2961317785545794),  
 ('space', 0.4428713694230737),  
 ('time', 0.1691181228294355),  
 ('took', 0.31098862362780794),  
 ('trying', 0.22698203563647698),  
 ('without', 0.220732770941426),  
 ('work', 0.1477740086182077),  
 ('worked', 0.2370572835239526)]
```

LDA Model - Each pair of lines represents a topic.
Each topic has a list of words representing it, along with their weights.

```
Score: 0.909971296787262  
Topic: 0.014*"amazonawscom" + 0.012*"png" + 0.011*"problem" + 0.010*"code" + 0.009*"try" + 0.008*"answer" + 0.007*"one" + 0.006*"exercise" + 0.006*"output" + 0.006*"name"  
  
Score: 0.010004638694226742  
Topic: 0.011*"print" + 0.011*"code" + 0.010*"function" + 0.010*"result" + 0.010*"string" + 0.008*"return" + 0.008*"test" + 0.007*"line" + 0.007*"range" + 0.006*"course"  
  
Score: 0.010003660805523396  
Topic: 0.010*"course" + 0.008*"code" + 0.008*"python" + 0.007*"problem" + 0.006*"hour" + 0.006*"line" + 0.006*"print" + 0.006*"time" + 0.006*"file" + 0.005*"great"  
  
Score: 0.010003485716879368  
Topic: 0.010*"line" + 0.010*"code" + 0.007*"return" + 0.007*"result" + 0.007*"problem" + 0.006*"need" + 0.006*"error" + 0.006*"true" + 0.006*"answer" + 0.006*"understand"  
  
Score: 0.01000332048654556  
Topic: 0.011*"problem" + 0.010*"code" + 0.009*"answer" + 0.007*"return" + 0.007*"help" + 0.006*"amazonawscom" + 0.006*"happens" + 0.006*"work" + 0.006*"browser" + 0.006*"tried"  
  
Score: 0.010003178380429745  
Topic: 0.010*"result" + 0.010*"code" + 0.009*"count" + 0.008*"return" + 0.007*"problem" + 0.007*"error" + 0.006*"line" + 0.006*"one" + 0.006*"yes" + 0.006*"value"  
  
Score: 0.01000300981104374  
Topic: 0.008*"code" + 0.008*"python" + 0.008*"error" + 0.007*"problem" + 0.007*"num" + 0.006*"course" + 0.006*"thats" + 0.006*"list" + 0.006*"still" + 0.006*"learn"  
  
Score: 0.010002578608691692  
Topic: 0.011*"code" + 0.010*"error" + 0.009*"problem" + 0.009*"result" + 0.009*"return" + 0.008*"amazonawscom" + 0.008*"line" + 0.007*"loop" + 0.007*"help" + 0.007*"true"  
  
Score: 0.010002501308918  
Topic: 0.011*"answer" + 0.010*"code" + 0.008*"exercise" + 0.008*"mysteryvalue" + 0.007*"problem" + 0.007*"question" + 0.006*"score" + 0.006*"line" + 0.006*"run" + 0.006*"variable"  
  
Score: 0.01000236626714468  
Topic: 0.009*"code" + 0.008*"problem" + 0.008*"answer" + 0.007*"dont" + 0.007*"return" + 0.006*"result" + 0.006*"course" + 0.006*"need" + 0.006*"price" + 0.006*"help"
```

Next Steps

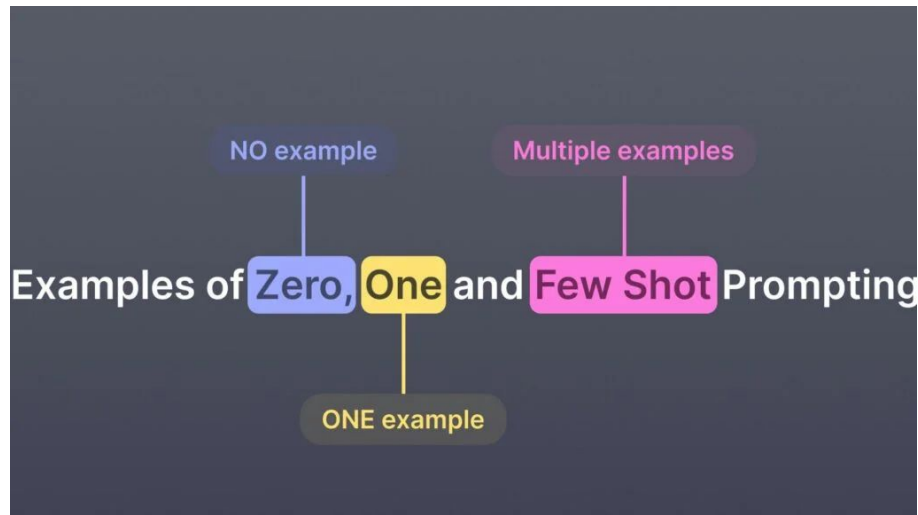
Outline for next steps:

1. **pyLDAvis:**
 - Create a visual representation that displays most relevant terms for each topic
2. **Explore alternatives for topic modeling**
 - Look into BERT and Naive Bayes
3. **Revamp the dashboard**

Machine Learning

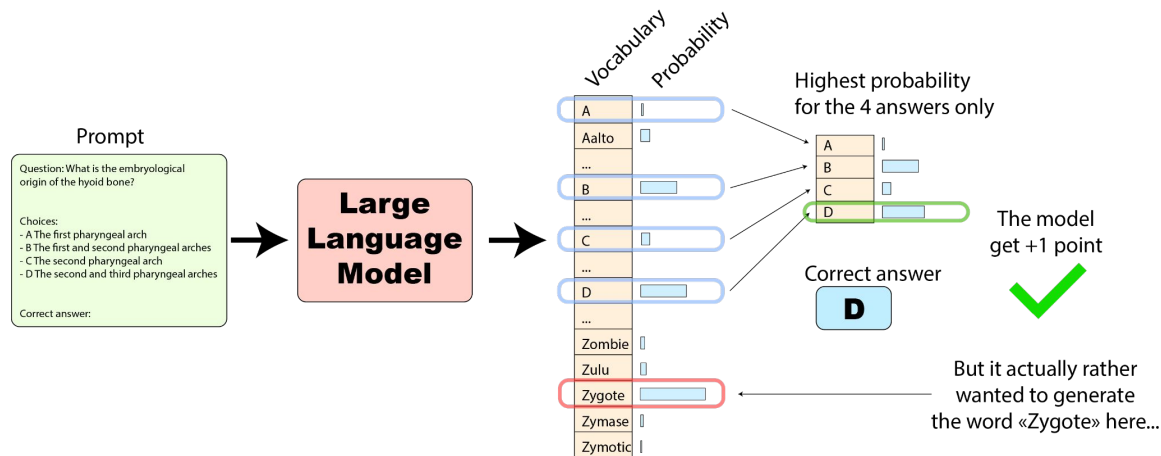
Prompt Refinement:

- Researched prompt engineering techniques:
 - No-shot: no examples
 - Few-shot: few examples
 - Chain-of-thought (CoT)
- Experimental Findings:
 - No-shot + CoT prompting provided the most unique response
- Integrated online MOOC data from last semester
- Added more contextual information to improve quality of dataset



Hugging Face Models

- Tested various models
- Explored the functionalities of each model based on its strengths
 - Found that certain models are incapable on generating human-readable text
- Chat-GPT responses: high-quality
 - Need to explore being able to call the API



Progress: Output & Code:

```
!pip install faker
!pip install transformers
!pip install torch

import pandas as pd
import random
import numpy as np
from faker import Faker
from transformers import AutoModelForCausalLM, AutoTokenizer
from IPython.display import display

# Initialize Faker to generate random names and other details
fake = Faker()

# Seed for reproducibility
random.seed(42)
np.random.seed(42)

# Load DistilGPT2 as a causal language model
model_name = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

# 1. Generate Student Information
def generate_students(num_students=20): # Generate 20 entries
    students = []
    for i in range(num_students):
        student_id = f"S{i+1:03}" # Unique ID like S001, S002, etc.
        name = fake.name()
        major = random.choice(["Computer Science", "Mechanical Engineering",
                               "Electrical Engineering", "Mathematics"])
        year = random.choice(["Freshman", "Sophomore"])
        students.append([student_id, name, major, year])
    return pd.DataFrame(students, columns=["StudentID", "Name", "Major", "Year"])
```

5

S006

Manuel Marsnall

Computer Science

Freshman

6

S007

Christopher Robertson

Mechanical Engineering

Freshman

7

S008

Peter Long

Computer Science

Freshman

8

S009

Mrs. Melissa McLaughlin

Mathematics

Freshman

9

S010

Katrina Meyer

Mathematics

Sophomore

10

S011

Ruth Carpenter

Computer Science

Freshman

11

S012

Ariana Wallace

Mathematics

Sophomore

12

S013

Michelle Pennington

Electrical Engineering

Freshman

13

S014

Linda Hill

Mechanical Engineering

Sophomore

14

S015

Shaun Edwards

Computer Science

Freshman

15

S016

Shannon Odonnell

Mathematics

Freshman

16

S017

Mia Miller

Electrical Engineering

Sophomore

17

S018

Thomas Bennett

Electrical Engineering

Freshman

18

S019

Christina Bailey

Mathematics

Freshman

19

S020

Dr. Vanessa Walker DDS

Mathematics

Freshman

Modules DataFrame:

	ModuleID	Topic	Subtopics	WeeksCovered	LearningOutcomes
0	M1	Variables and Data Types	[Integers, Strings, Booleans]	1	Understand basic data types and variables
1	M2	Loops and Conditionals	[For Loops, If Statements]	2	Use loops for repetition and conditions
2	M3	Functions	[Defining Functions, Parameters]	2	Define and use functions in Python
3	M4	Data Structures	[Lists, Dictionaries]	2	Use basic data structures
4	M5	File I/O	[Reading Files, Writing Files]	1	Read from and write to files in Python

Assignments DataFrame:

	AssignmentID	ModuleID	AssignmentTopic	DueDate	MaxScore	AverageScore
0	A1	M1	Variables and Data Types Practice	2024-12-31	100	87
1	A2	M2	Loops and Conditionals Practice	2024-05-28	100	79

Next Steps for ML:

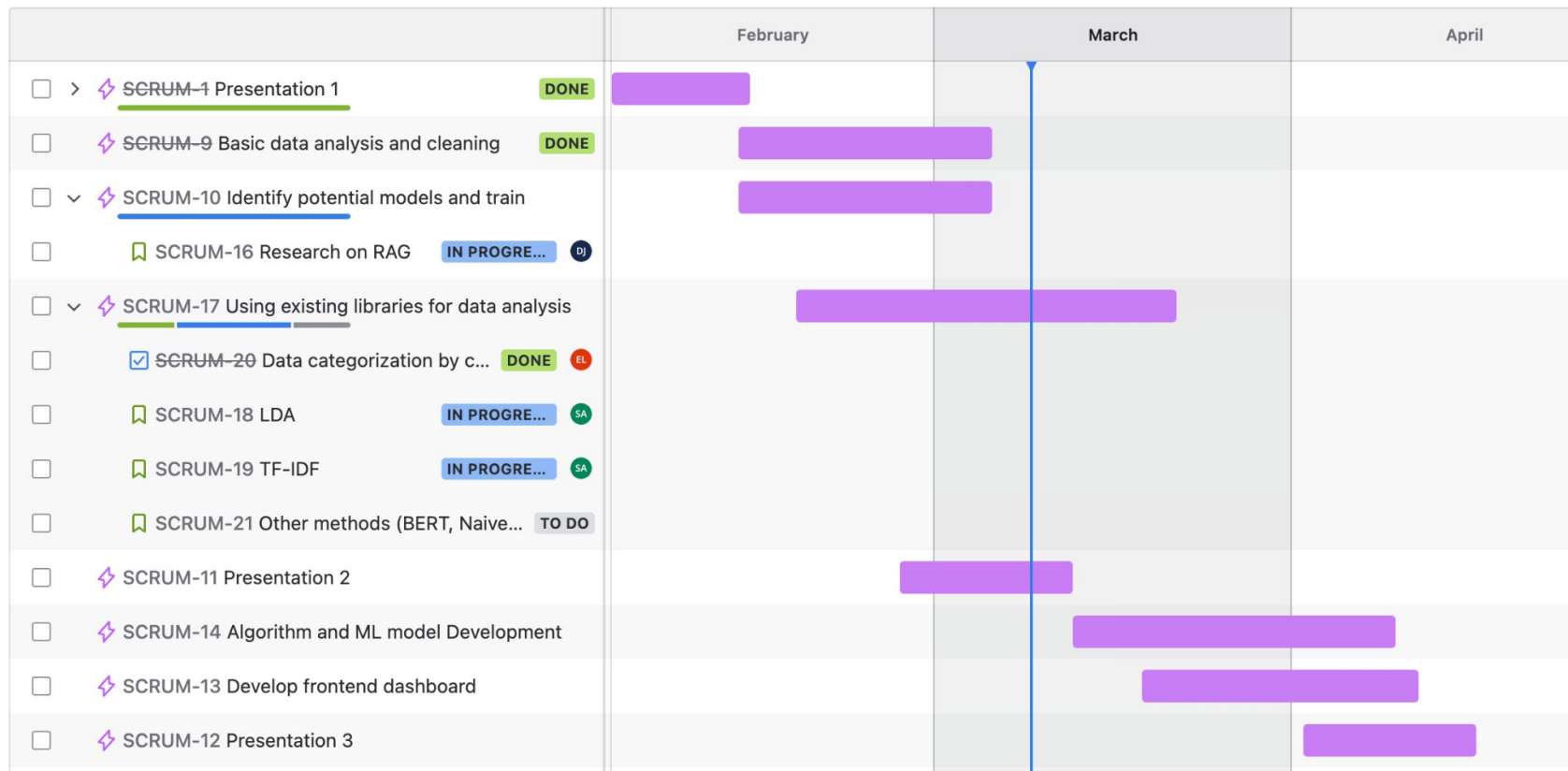
- Leverage RAG research to integrate it
- Research out to AI Education subteam
- Reach out to the Harvard Researcher that presented last semester
- Explore llama and Claude models
- Try to get access to the Chat-GPT API
- Refine the prompt more



Conclusion

- Completed the basic data preparation work
- Used existing libraries for data analysis and topic modeling
 - LDA and TF-IDF
- Researched on machine learning techniques
- TODO for next sprint
 - Research more fields of dataset
 - Research on other possible methods and implement basic backend

Project Timeline Update



Citations

- Farshad, S., Zorin, E., Amangeldiuly, N., & Fortin, C. (2024). Engagement Assessment in Project-Based Education: A Machine Learning Approach in Team Chat Analysis. *Education and Information Technologies*, 29, 13105-13131. <https://doi.org/10.1007/s10639-023-12381-5>
- Galán Hernández, J. J., Marín Díaz, G., & Mariscal, G. (2024). Methodology for analyzing educational forums with NLP: Searching for economic terms. In *Teaching innovations in economics* (pp. 77–97). Springer. https://doi.org/10.1007/978-3-031-72549-4_4
- Jelodar, H., Wang, Y., Orji, R., & Huang, S. (2020). Deep Sentiment Classification and Topic Discovery on Novel Coronavirus or COVID-19 Online Discussions: NLP Using LSTM Recurrent Neural Network Approach. *IEEE Journal of Biomedical and Health Informatics*, 24(10), 2733-2742.
- Liu, Sannyuya, et al. "Automated Detection of Emotional and Cognitive Engagement in MOOC Discussions to Predict Learning Achievement." *Computers and Education*, vol. 181, 2022, pp. 104461-, <https://doi.org/10.1016/j.compedu.2022.104461>.
- Mello, R. F., Freitas, E., Cabral, L., Pereira, F. D., Rodrigues, L., Rakovic, M., Raniel, J., & Gašević, D. (2024). Words of wisdom: A journey through the realm of natural language processing for learning analytics—A systematic literature review. *Journal of Learning Analytics*, 11(3), 82–105. <https://doi.org/10.18608/jla.2024.8403>
- Zou, W., Hu, X., Pan, Z., Li, C., Cai, Y., & Liu, M. (2021). Exploring the relationship between social presence and learners' prestige in MOOC discussion forums using automated content analysis and social network analysis. *Computers in Human Behavior*, 115, 106582. <https://doi.org/10.1016/j.chb.2020.106582>

Thanks for listening!

Q&A